

OpenText™ Identity Manager

Formio Help

Version : 25.4

PDF Generated on : April 21, 2026

Table of Contents

[1. Formio Help](#)

1

1. Formio Help

Tree Element

The Tree element is used to design a hierarchy or a category. There are options to configure nodes that allow you to change the design of the form in the desired format.

The screenshot shows the 'TREE ELEMENT COMPONENT' configuration window. It has tabs for 'Display', 'Data', 'Validation', 'API', 'Conditional', and 'Logic'. The 'Data' tab is active. Below the tabs, there's a 'Layout' section and a 'Data Source Type' dropdown set to 'JSON'. The 'Data Source Raw JSON' field contains a JSON structure with a root object containing 'dn', 'name', 'data', and 'subContainers' (an array of objects). Below this, the 'Mapper keys for URL response/ Raw JSON' field shows a mapping object. The 'Default Value' field is set to 'Default Value'. The 'Refresh On' dropdown is empty. There's a 'Clear Value On Refresh' checkbox. On the right, a 'Preview' section shows a 'Tree Element' with a single node. At the bottom right of the preview are 'Save', 'Cancel', and 'Remove' buttons.

Tree Element - Data



Note

The **Preview** area displays how the form would render after making the changes or edits to any field.

The following fields are populated in the **Data** tab.

Data Source Type: Select the type of data source, that is JSON or URL.

The following fields appear when the **Data Source Type** is **JSON**:

Data Source Raw JSON: Each node configuration is displayed in a JSON file. You must have the following details mentioned:

- **dn:** It is a unique ID that defines the selected node. The value must be a string. This is the data value for the tree element you add, and is passed to the workflow.
- **name:** Enter the name you wish to be displayed on the User Interface. The value must be a string.
- **data:** (Optional) This saves the metadata to the existing field.
- **subcontainers:** It defines the child node. The value must be an array of JSON. Each element of the array is an object following the same structure as that of a node.
- **icon:** The image class to display an icon on each node. The value must be a string. If you do not define the value, the node uses the image class selected in **Default Icon Class** under the **Display** tab.



Note

All nodes must follow either JSON or a customized data structure. The parent and child cannot be of different structures.

If JSON does not follow the default structure, it must be mapped accordingly.

Mapper keys for URL response: Allows you to map the JSON parameters with the URL element parameters.

You can use the **Mapper keys for URL response/ Raw JSON** setting to customize the field names provided under **Data Source RAW JSON**.

For example, if you want the field names as ["id","display","description","child","icon"], then modify the details under **Mapper keys for URL response/ Raw JSON** as follows:

```
{ dn: "id", name: "display", data: "description", subContainers: "child", icon: "icon"}
```

The following fields appear when the **Data Source Type** is **URL**:

HTTP method: Select the required http method. For example, GET, POST.

Service ID: Select the appropriate service ID. For example, IDM, IG.

Data Source URL: The URL that returns a JSON array to use as the data source.

Mapper keys for URL response: Allows you to map the JSON parameters with the URL element parameters.

Request Headers: Set any headers that should be sent along with the request to the URL. This is useful for authentication. You can have multiple keys for a parameter and multiple values for a key.

request payload: Enter the request body for the root node. The request body contains the node details that will be used to load the node. It applies in the case of the POST method.

lazy parameter: Enter the parameter name that will be used to load a sub-node. The lazy parameter, such as `nodeid` is appended in the data source URL provided by the user. It applies in case of the GET method (Lazy Loading).

Default Value: The entered value is displayed in the field before user interaction. Having a default value overrides the placeholder text.

Refresh On: Refreshes data when another field changes.

Clear Value On Refresh: This text appears below the input field.

Dynamic Entity

This is a multi-select drop-down field. This widget allows you to select more than one entity in an entity type. For example, user, group

The screenshot shows the 'Dynamic Entity Component' configuration window. It has tabs for 'Display', 'Data', 'Validation', 'API', 'Conditional', and 'Logic'. The 'Display' tab is active. The configuration includes:

- Entity Type:** A text input field with the placeholder 'Enter Entity Type'.
- Display Attribute:** A text input field with the placeholder 'Enter Display Attribute' and a '+ Add Another' button.
- Service ID:** A dropdown menu currently showing 'IDM'.
- Limit:** A text input field with the value '20' and a note: 'Maximum number of items to view per page of results.'
- Default Value:** A text input field with the placeholder 'Default Value'.
- Refresh On:** A dropdown menu.
- Clear Value On Refresh:** A checkbox that is currently unchecked.
- + Custom Default Value:** A button.

On the right side, there is a 'Preview' section showing a 'Dynamic Entity' label above a multi-select dropdown field. Below the preview are 'Save', 'Cancel', and 'Remove' buttons.

Dynamic Entity

The following fields are populated in the **Data** tab:

Entity Key: Specify the entity key for the entity type. For example, for the User entity type, the entity key is user.

Display Expression Attribute: Specify the attributes of the entity type you want to be displayed. You can add multiple display attributes. For example, for the entity type User, the **Display Expression Attribute** can be FirstName, LastName.

Entity Type	Display Expression Attribute
User	FirstName, LastName

Service ID: Select the service ID. For example, OpenText Identity Lifecycle Manager or OpenText Identity Governance.

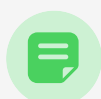
Limit: Specify the number of entities that you want to be displayed. The default value is 20.

Default Value: Specify the value to be displayed in the field before user interaction. Having a default value overrides the placeholder text.

Refresh On: Refreshes data when another field changes.

DN Query

Allows you to search and retrieve DNs from the Identity Vault. However, with the DNQuery, the object selector content can be driven by the result of a directory abstraction layer Queries object rather than from properties.



Note

You must ensure that the query, parameter, and key you specify is present in Designer.

The screenshot shows the 'DN Query Component' configuration window. It features a top navigation bar with tabs: Display, Data, Validation, API, Conditional, and Logic. The 'Data' tab is selected. Below the tabs, there's a 'Service ID' dropdown menu currently showing 'IDM'. A 'Parameters' section contains a table with 'Key' and 'Value' columns, and an '+ Add Another' button. Below this is a 'Query Key' text input field. A 'Return Attributes' section has a text input field and another '+ Add Another' button. At the bottom, there's a 'key of return parameter' label. On the right side, a 'Preview' panel displays the 'DN Query' field and three buttons: 'Save' (green), 'Cancel' (grey), and 'Remove' (red).

DN Query

The following fields are populated in the **Data** tab:

Service ID: Select the required service ID. For example, OpenText Identity Lifecycle Manager or OpenText Identity Governance.

Parameters: Specify the parameter key and its value. You can have multiple keys for a parameter, and you can have multiple values for a key.



Note

You must provide the parameter value (static value) while configuring this component. Dynamic parameter value such as `data.<other dynamic value>` is not supported. To use the dynamic parameter value, you can use the `IDVault.globalquery` in the **Select** component.

Query Key: Specify the key of the DAL Queries object.

Return Attributes: Specify the attributes of the entity type you want to be displayed. You can add multiple display attributes.

Default Value: Specify the value to be displayed in the field before user interaction. Having a default value overrides the placeholder text.

Refresh On: Refreshes data when another field changes.

Title Element

This widget is used to design a title for the form.

The screenshot shows the 'Title Element Component' configuration window. It has a tabbed interface with 'Display' selected. The configuration fields include:

- Label:** A text field containing 'Title Element'.
- Hide Label:** A checkbox that is currently unchecked.
- Label Position:** A dropdown menu set to 'Top'.
- Title Content:** A text field containing 'Title'.
- CSS Class:** A text field containing 'CSS Class'.
- Attributes:** A table with two columns: 'Attribute' and 'Value'. It is currently empty.
- + Add Another:** A blue button to add more attributes.

 On the right, there is a 'Preview' section showing a blue 'Title' label. Below the preview are 'Save' (green), 'Cancel' (grey), and 'Remove' (red) buttons. A 'Help' icon is in the top right corner.

Title Element

DN Display

This is used to display a read-only DN. It can display the full DN or a set of attributes associated with the DN depending on the properties you set.

The screenshot shows the 'DN Display Component' configuration window. It has a tabbed interface with 'Display' selected. The configuration fields include:

- Entity key for DN Expression:** A text field containing 'Entity key for DN Expression'.
- Display Expression:** A text field containing 'Display Expression'.
- DN:** A text field containing 'Enter Fully Qualified DN'.
- Default Value:** A text field containing a colon ':'.
- Refresh On:** A dropdown menu.
- Clear Value On Refresh:** A checkbox that is currently unchecked.
- + Custom Default Value:** A button to add a custom default value.
- + Calculated Value:** A button to add a calculated value.
- Allow Manual Override of Calculated Value:** A checkbox that is currently unchecked.
- Encrypt:** A checkbox that is currently unchecked.
- Database Index:** A checkbox that is currently unchecked.

 On the right, there is a 'Preview' section showing 'DN Display :'. Below the preview are 'Save' (green), 'Cancel' (grey), and 'Remove' (red) buttons. A 'Help' icon is in the top right corner.

DN Display

The following fields are populated in the **Data** tab:

Entity key for DN Expression: Represents the type of entity used for DN display.

Leave this value blank if you want to display the full DN or CN value retrieved from the Identity Vault; else enter an entity.

The entity you choose must:

- Have the directory abstraction layer View property set to **True**.
- Be the entity of the DN you are working with.

Display Expression: Represents the attribute used for DN display. It also displays multiple attributes, provided the attribute is separated by a comma. Leave this value blank if you want to display the full DN or CN value. If you want to mask the DN by displaying attributes, you must first specify an **Entity key for DN expression**.

For example, to show the user entity's first and last name attributes, construct an expression like this: `FirstName LastName`.

Ensure the attribute's View, Read, Search, and Required properties are set to True in the directory abstraction layer. For more information, see [Attribute Properties](#) in the [Administrator's Guide to Designing the Identity Applications](#).

DN: Enter the respective DN.

Default Value: The entered value is displayed in the field before user interaction. Having a default value overrides the placeholder text.

Refresh On: Refreshes data when another field changes.

Clear Value On Refresh: When the **Refresh On** field is changed, clear this component's value.

Permission Request DN

This is used to provide permission for a default role and resource approval template.

The screenshot shows the 'Permission Request DN Component' configuration window. It has a 'Display' tab selected. The 'Permission Request Type' dropdown is empty. The 'Hide Label' checkbox is unchecked. The 'Label Position' dropdown is set to 'Top'. The 'placeholder' field contains 'placeholder'. The 'Description' field contains 'Description for this field.'. The 'Tooltip' field contains 'To add a tooltip to this field, enter text here.'. The 'Error Label' field contains 'Error Label'. The 'Custom CSS Class' field contains 'Custom CSS Class'. On the right, there is a 'Preview' section with an empty input field and 'Save', 'Cancel', and 'Remove' buttons.

Permission Request DN

In the **Display** tab, based on the selected **Permission Request type**, the label is automatically populated.

Label Element

This widget allows you to create and design the labels used in the forms.

The screenshot shows the 'Label Element Component' configuration window. It has a 'Display' tab selected. The 'Label' field contains 'Label'. The 'Hide Label' checkbox is unchecked. The 'Label Position' dropdown is set to 'Top'. The 'Label Width' field contains '30' with a '%' button. The 'Label Content' field contains 'Label Content'. The 'Widget' dropdown is set to 'Select a widget'. The 'placeholder' field contains 'placeholder'. The 'Description' field contains 'Description for this field.'. On the right, there is a 'Preview' section showing 'Label' followed by a colon and a space, and 'Save', 'Cancel', and 'Remove' buttons.

Label Element

Data Item Mapping Element

This widget allows you to map data from the data flow into fields in a form (pre-activity mapping) and to map data from the form back to the data flow (post-activity mapping). If you have specified the Data Item Mapping Value in OpenText Designer, the **Custom Default Value** will overwrite the **Data Mapping Value**. In other words, the **Custom Default Value** takes precedence.

Data Item Mapping Element



© Copyright 2026 Open Text

For more info, visit <https://docs.microfocus.com>
